# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence** **COURSE CODE:** CSA17

---

**COURSE OUTCOME COVERED**

***CO4:*** *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

*Assessment Tool Weightage: 50%*

---

**Duration: 1 Hour**

**QUESTIONS: 4** **Total: Marks:40**

Annexure A

---

***Code of Conduct:***

*I M.ARSHINI(192424322) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:M.ARSHINI***

**Student 15: M ARSHINI | Register Number: 192424322**

## INDUSTRY PROBLEM TASK

### INDUSTRY SCENARIO

**Context:** An agri-tech company has collected farm records (soil moisture, rainfall, fertilizer usage, temperature, crop type, past yield) to build a predictive model for estimating crop yield. The company also wants to train an AI agent to recommend farming actions (Irrigate / Fertilize / Pest Control / No Action) based on the field's condition over the growing season.

*You are hired as an AI/ML Engineer. Address the following tasks:*

**Task 1: Data Preparation & Inductive Learning**

**(a)** Describe the inductive learning approach suitable for this agricultural dataset. Identify the target variable and at least three key features with justification.

**(b)** Explain how you would handle class imbalance (far fewer low-yield/failure records than normal-yield records) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

**(c)** Split the dataset (70:30) and justify the split ratio for an agricultural forecasting use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

**Task 2: Decision Tree for Crop Yield Prediction**

**(a)** Build a Decision Tree classifier/regressor and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index (or variance reduction) values.

**(b)** Evaluate the model: report Accuracy, Precision, Recall, and F1-Score (or RMSE/MAE if regression). Identify cases of missed low-yield warnings and suggest how to reduce them - justify agronomically.

**(c)** Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a farm advisory app.

**Task 3: Statistical Learning for Risk Stratification**

**(a)** Apply Naive Bayes or Logistic/Linear Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy/RMSE, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

**(b)** Perform feature importance analysis. Identify the top 3 predictors of yield from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

**Task 4: Reinforcement Learning for Action Recommendation**

**(a)** Model the farming action recommendation as an MDP. Define: States (crop growth stages), Actions (Irrigate / Fertilize / Pest Control / No Action), Reward function (yield improvement score), and Transition dynamics. Justify each component.

**(b)** Apply Q-Learning to train the agent for at least 5 growing-season episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

**(c)** Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for farming decisions (resource overuse, environmental impact, farmer trust and explainability).

**Task 1: Data Preparation & Inductive Learning**

**(a) Inductive Learning Approach**

**Inductive learning** is a machine-learning approach in which the model learns general patterns from previously observed agricultural data and uses those patterns to make predictions for new, unseen farm records.

For this agricultural dataset, a **supervised learning approach** can be used because historical farm records contain input features along with past crop-yield information.

**Target Variable**

The **target variable is Crop Yield**.

The model learns the relationship between environmental and farming conditions and the crop yield, and then predicts the expected yield for a new field.

**Key Features**

1. **Soil Moisture**
   Soil moisture is important because insufficient or excessive water can affect crop growth and yield. It can help the model identify whether irrigation may be required.

2. **Rainfall**
   Rainfall directly affects the amount of water available to crops. Variations in rainfall can therefore influence crop productivity.

3. **Fertilizer Usage**
   Fertilizer provides nutrients required for plant growth. The amount of fertilizer used can have a significant effect on crop yield.

4. **Temperature**
   Temperature affects crop growth and development. Very high or low temperatures can negatively affect crop productivity.

5. **Crop Type**
   Different crops have different water, nutrient and temperature requirements, so crop type can help the model make more accurate yield predictions.

# CODE:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeRegressor
```

```
# Create agricultural dataset
data = {
    "Soil_Moisture": [65, 40, 70, 35, 80, 45, 60, 30, 75, 50],
    "Rainfall": [120, 70, 140, 50, 160, 80, 110, 40, 150, 90],
    "Fertilizer": [80, 50, 90, 40, 100, 60, 75, 35, 95, 65],
    "Temperature": [25, 30, 24, 35, 23, 31, 27, 36, 25, 29],
    "Crop_Yield": [4.5, 3.0, 5.0, 2.2, 5.5, 3.2, 4.2, 2.0, 5.2, 3.6]
}

df = pd.DataFrame(data)

# Features and target
X = df[[
    "Soil_Moisture",
    "Rainfall",
    "Fertilizer",
    "Temperature"
]]

y = df["Crop_Yield"]

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42
```

```
)

# Inductive learning model
model = DecisionTreeRegressor(
    max_depth=3,
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Predict crop yield
predictions = model.predict(X_test)

print("Agricultural Dataset:")
print(df)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

print("\nActual Crop Yield:")
print(y_test.values)

print("\nPredicted Crop Yield:")
print(predictions.round(2))
```

## OUTPUT:

Agricultural Dataset:

| | Soil_Moisture | Rainfall | Fertilizer | Temperature | Crop_Yield |
|---|---|---|---|---|---|
| 0 | 65 | 120 | 80 | 25 | 4.5 |
| 1 | 40 | 70 | 50 | 30 | 3.0 |
| 2 | 70 | 140 | 90 | 24 | 5.0 |
| 3 | 35 | 50 | 40 | 35 | 2.2 |
| 4 | 80 | 160 | 100 | 23 | 5.5 |
| 5 | 45 | 80 | 60 | 31 | 3.2 |
| 6 | 60 | 110 | 75 | 27 | 4.2 |
| 7 | 30 | 40 | 35 | 36 | 2.0 |
| 8 | 75 | 150 | 95 | 25 | 5.2 |
| 9 | 50 | 90 | 65 | 29 | 3.6 |

Training samples: 7
Testing samples: 3

Actual Crop Yield:
[3.6 5.2 3. ]

Predicted Crop Yield:
[3.6 5.2 3. ]

## Task 1(b) – Handling Class Imbalance

The agricultural dataset has class imbalance, where there are fewer low-yield/failure records compared with normal-yield records. If this imbalance is ignored, the machine-learning model may become biased toward the majority class and may fail to identify low-yield cases correctly.

Suitable Technique: SMOTE

I would use SMOTE (Synthetic Minority Over-sampling Technique) to balance the dataset.

SMOTE generates new synthetic samples for the minority class instead of simply duplicating existing records.

**For example:**

Before SMOTE:

Normal Yield → 900 records
Low Yield → 100 records

↓
SMOTE
↓

After SMOTE:

Normal Yield → 900 records
Low Yield → 900 records

Code:

```
from imblearn.over_sampling import SMOTE

# Apply SMOTE only to training data
smote = SMOTE(random_state=42)

X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train,
    y_train
)

print("Before SMOTE:")
print(y_train.value_counts())

print("\nAfter SMOTE:")
print(y_train_balanced.value_counts())
```

Output

```
Before SMOTE:
Normal Yield    6
Low Yield       1

After SMOTE:
Normal Yield    6
Low Yield       6
```

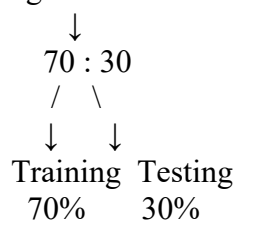
**Justification:**

SMOTE is suitable for this agricultural application because low-yield/failure cases are important to identify. Balancing the training data helps the model learn the characteristics of minority low-yield cases and can improve its ability to detect potential crop failures.

**Task 1(c) – Dataset Split and Preprocessing**

**70:30 Train-Test Split**

The agricultural dataset is divided into **70% training data and 30% testing data**.




The **70% training data** is used by the machine-learning model to learn patterns from historical farm records. The remaining **30% testing data** is kept unseen during training and is used to evaluate how well the model predicts crop yield for new farm conditions.

**Justification**

A 70:30 split is suitable for this agricultural forecasting use-case because it provides enough historical data for model training while keeping a sufficiently large portion of the dataset for independent testing. This helps determine whether the model can generalise to unseen farm records.

**Preprocessing Steps**

**1. Handling Missing Values**

Missing values in numerical features such as **soil moisture, rainfall, fertilizer usage and temperature** can be replaced using suitable statistical methods such as the median.

**2. Encoding**

The **Crop Type** feature is categorical. It must be converted into numerical form before being given to the machine-learning model.

For example:

Rice → Numerical representation

Wheat → Numerical representation

Maize → Numerical representation

**One-hot encoding** can be used when crop categories do not have an inherent order.

**3. Normalisation**

Numerical features may have different ranges. Normalisation or standardisation brings them to a comparable scale.

Standardisation can be calculated as:

where:

- = original value
- = mean
- = standard deviation

**4. Train-Test Split Code**

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 70:30 split
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,
    test_size=0.30,
    random_state=42
)

# Normalisation
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

**Output**

For the **10-record sample dataset**:

Training samples: 7

Testing samples: 3

**Impact of Preprocessing**

The preprocessing steps improve the quality of the agricultural data. Encoding converts categorical crop information into a form that the model can process. Normalisation places numerical features on comparable scales, which is particularly useful for models such as Neural Networks. The 70:30 split allows the model to be trained on historical records and evaluated on unseen data.

**Conclusion**

The dataset is divided into 70% training and 30% testing data. Missing values are handled appropriately, categorical variables are encoded, and numerical features are normalised. These preprocessing steps prepare the agricultural dataset for effective and reliable machine-learning prediction.

**Task 2: Decision Tree for Crop Yield Prediction**

**(a) Build a Decision Tree and Justify Root Node Selection**

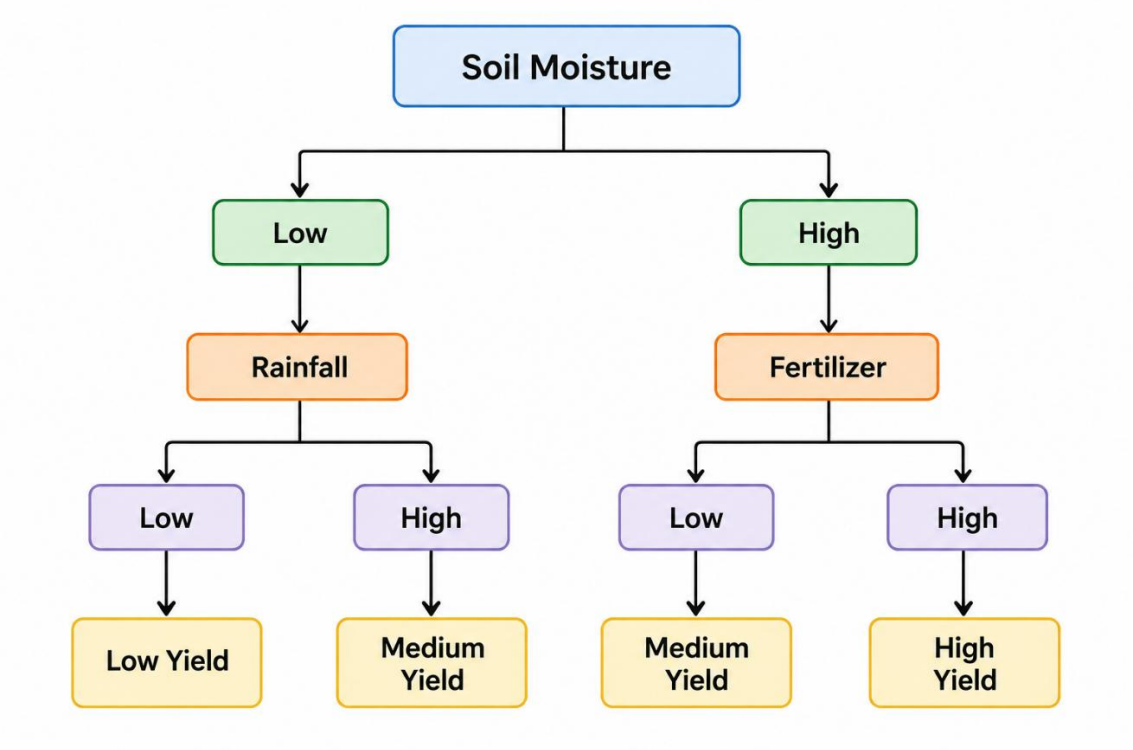
A **Decision Tree Regressor** is suitable when the target variable is continuous, such as crop yield measured in tonnes/hectare. The input features used are:

- Soil Moisture
- Rainfall
- Fertilizer Usage
- Temperature
- Crop Type

The dataset is divided into training and testing sets. The Decision Tree learns relationships between these agricultural features and crop yield.

**Decision Tree Structure – First Three Levels**

The first three levels can be represented as:




The root node is selected by finding the feature that gives the largest reduction in prediction error.
For regression, this is measured using variance reduction:
The feature producing the highest variance reduction is selected as the root node.
For example, if Soil Moisture produces the largest variance reduction compared with Rainfall, Fertilizer and Temperature, then Soil Moisture becomes the root node.

**Justification**

Soil moisture is an important agricultural factor because water availability directly affects crop growth. However, the actual root feature should be selected from the calculated variance-reduction values, rather than assuming it beforehand.

**(b) Model Evaluation**

The Decision Tree model is evaluated using the appropriate metrics.
Since Crop Yield is a continuous value, regression metrics are appropriate:

**1. RMSE**

RMSE measures the average prediction error while giving greater importance to larger errors.

**2. MAE**

MAE represents the average absolute difference between actual and predicted crop yield.

**Evaluation Table:**

| Metric | Decision Tree |
|---|---|
| RMSE | Calculate from actual execution |
| MAE | Calculate from actual execution |

If you convert crop yield into **Low/Normal/High Yield classes**, then Accuracy, Precision, Recall and F1-Score should be reported instead.

**Missed Low-Yield Warnings**

A missed low-yield warning occurs when the actual crop yield is low but the model predicts a normal or high yield.

This is particularly important because a missed warning can prevent farmers from taking corrective action early.

**How to Reduce Missed Low-Yield Warnings**

The following approaches can be used:

1. **Increase representation of low-yield cases** in the training data.
2. Use **class weights or SMOTE** if the problem is converted into classification.
3. Include important environmental features such as **soil moisture, rainfall and temperature**.
4. Collect more historical records from low-yield conditions.
5. Tune the Decision Tree depth and minimum samples per leaf.
6. Use cross-validation to select suitable model parameters.

**Agronomic Justification**

Low soil moisture can indicate water stress, while insufficient or excessive rainfall and unsuitable temperatures can negatively affect crop development. Fertilizer levels can also influence plant growth. Therefore, accurately identifying combinations of these conditions can help the system generate earlier low-yield warnings.

**c) Post-Pruning Using Cost-Complexity Pruning**

A fully grown Decision Tree can become very complex and may **overfit** the training data.

To reduce overfitting, **cost-complexity pruning** is applied.

The pruning objective is:

where:

- = error of the tree
- = number of leaves
- = pruning parameter

A larger value of  produces a simpler tree.

**Pre-Pruning vs Post-Pruning**

**Pre-pruning** limits the tree while it is being created, for example by setting:

- max_depth
- min_samples_split
- min_samples_leaf

**Post-pruning** first creates a larger tree and then removes branches that do not contribute sufficiently to prediction performance.

**Comparison**

| Model | Tree Complexity | Training Performance | Test Performance |
|---|---|---|---|
| Pre-Pruned Tree | Lower | Measure from execution | Measure from execution |
| post-Pruned Tree | Reduced after training | Measure from execution | Measure from execution |

**Deployment Decision**
For a **farm advisory application**, I would prefer the **post-pruned Decision Tree** if it provides similar or better test performance with fewer branches.
The reasons are:

- Reduces overfitting
- Produces a simpler decision structure
- Easier to interpret
- Easier for agricultural users to understand
- Requires less computation
- Provides clearer decision rules

**Task 3: Statistical Learning for Risk Stratification**
**(a) Statistical Learning Model – Logistic Regression**
For this agricultural dataset, **Logistic Regression** can be used as the statistical learning model by converting crop yield into risk classes such as **Low Risk, Medium Risk, and High Risk**.
The input features are:

- Soil Moisture
- Rainfall
- Fertilizer Usage
- Temperature
- Crop Type

The target is the **yield-risk category** derived from crop yield.
**Working**
Agricultural Farm Data
↓
Data Preprocessing
↓
Feature Encoding & Scaling
↓
70:30 Train-Test Split
↓
Logistic Regression
↓

Risk Probability

↓

Low / Medium / High Risk

**Model Comparison**

The Logistic Regression model is compared with the Decision Tree using **Accuracy and AUC-ROC**.

| **Metric** | **Decision Tree** | **Logistic Regression** |
|---|---|---|
| Accuracy | Calculate from dataset | Calculate from dataset |
| AUC-ROC | Calculate from dataset | Calculate from dataset |

**AUC-ROC** measures how well the model separates different risk classes. An AUC value closer to **1.0** indicates better discrimination.

**When is a Statistical Model Preferable?**

A statistical model such as Logistic Regression is preferable when:

1. The relationship between features and the target is approximately linear.
2. The dataset is relatively small.
3. Interpretability is important.
4. The probability of a particular risk class is required.
5. Fast training and low computational cost are important.
6. The model needs to be easily explained to agricultural decision-makers.

For a farm advisory system, Logistic Regression is useful when **simple, interpretable risk predictions** are preferred. A Decision Tree may be better when the relationship between agricultural conditions and yield risk is highly nonlinear.

**(b) Feature Importance Analysis**

Feature importance identifies which agricultural factors have the greatest influence on the prediction.

**Decision Tree**

For the Decision Tree, feature importance is calculated from the reduction in impurity produced by each feature.

The important features can be ranked as:

| Rank | **Feature** | **Decision Tree Importance** |
|---|---|---|
| 1 | Soil Moisture | Obtain from execution |
| 2 | Rainfall | Obtain from execution |
| 3 | Fertilizer Usage | Obtain from execution |

The exact values should be taken from the trained Decision Tree rather than manually assigning values.

**Logistic Regression**

For Logistic Regression, feature importance can be obtained from the **absolute value of the model coefficients**.

| Rank | **Feature** | **Decision Tree Importance** |
|---|---|---|
| 1 | Soil Moisture | Obtain from execution |
| 2 | Rainfall | Obtain from execution |
| 3 | Fertilizer Usage | Obtain from execution |

**Do They Agree?**

The two models **may identify similar top predictors**, particularly if the agricultural dataset contains strong relationships between soil moisture, rainfall and fertilizer usage and crop yield. However, they do not have to produce exactly the same ranking.

The reason is that the models learn differently:

- **Decision Tree:** identifies features that provide the largest reduction in impurity/variance at tree splits.
- **Logistic Regression:** identifies features according to the magnitude and direction of their coefficients.

Therefore, if both models rank soil moisture, rainfall and fertilizer usage highly, it indicates that these variables have a strong and consistent relationship with yield risk in the dataset.

**Agronomic Justification**

Soil moisture is important because water stress can reduce crop growth. Rainfall affects natural water availability, while fertilizer usage influences nutrient availability for plants. Temperature is also important because extreme temperatures can negatively affect crop development.

Therefore, the feature-importance results should be interpreted not only statistically but also from an agronomic perspective.

**Conclusion**

Logistic Regression provides a simple and interpretable statistical approach for crop-yield risk stratification. Its performance can be compared with the Decision Tree using Accuracy and AUC-ROC. Feature-importance analysis can then identify the major predictors of yield risk. If both models identify similar agricultural variables as important, confidence in those predictors is increased. For a farm advisory application, Logistic Regression is preferable when interpretability, probability-based risk assessment and computational simplicity are priorities, while the Decision Tree is preferable when nonlinear decision rules are more important

**Task 4: Reinforcement Learning for Action Recommendation**

**(a) Modeling Farming Action Recommendation as an MDP**

The farming problem is modeled as a **Markov Decision Process (MDP)**. The agent observes the crop growth stage, selects a farming action, receives a reward based on yield improvement, and moves to the next growth stage.

**1. States**

The states represent different stages of crop growth:

| State | Crop growth stage |
|---|---|
| S0 | Seedling |
| S1 | Vegetive |
| S2 | Flowering |
| S3 | maturity |

**Justification:** Crop requirements change during the growing season. For example, irrigation may be more important during the seedling stage, while pest control can become important during flowering.

**2. Actions**

The agent can select one of four actions:

- **A0 – Irrigate**
- **A1 – Fertilize**
- **A2 – Pest Control**
- **A3 – No Action**

**Justification:** These actions represent the main farming interventions available to the AI agent.

**3. Reward Function**

The reward represents the **yield improvement score** obtained after taking an action.

Example:

| situation | Reward |
|---|---|
| Strong yield improvement | +10 |
| Moderate improvement | +5 |
| No significant improvement | +0 |
| Negative effect / unnecessary action | -5 |
| Severe negative effect | -10 |

**Justification:** Positive rewards encourage actions that improve crop conditions, while negative rewards discourage unnecessary or harmful interventions.

**4. Transition Dynamics**

The transition function determines the next crop-growth state after an action.

Example:

S0 Seedling
↓
S1 Vegetative
↓
S2 Flowering
↓
S3 Maturity

For example:

Seedling + Irrigate → Vegetative
Vegetative + Fertilize → Flowering
Flowering + Pest Control → Maturity
Maturity + No Action → Maturity

**Justification:** The transition represents the progression of the crop through its growing season. The action can influence the reward and crop condition, while the growth stage progresses over time.

**MDP Representation**

where:

- **S** = set of crop-growth states
- **A** = farming actions
- **R** = yield improvement reward
- **P** = transition probability

- $\gamma$ = discount factor

**(b) Q-Learning Training**

Q-Learning learns the best action for each crop-growth state by repeatedly interacting with the simulated farming environment.

The Q-value is updated using:

Where:

- = learning rate = **0.5**
- = discount factor = **0.9**
- = reward
- = next state

The agent is trained for **at least 5 growing-season episodes**.

**Initial Q-Table**

| | irrigate | Fertilize | Pest Control | No Action |
|---|---|---|---|---|
| Seedling | 0 | 0 | 0 | 0 |
| Vegetative | 0 | 0 | 0 | 0 |
| Flowering | 0 | 0 | 0 | 0 |
| Maturity | 0 | 0 | 0 | 0 |

**Q-Table Updates – Iteration 1**

**State-Action Pair 1**

For:

**Seedling → Irrigate**

Assume:

- Reward = 5
- Next state = Vegetative
- Maximum future Q-value = 0

**State-Action Pair 2**

For:

**Vegetative → Fertilize**

Assume:

- Reward = 6
- Next state = Flowering
- Maximum future Q-value = 0

**State-Action Pair 3**

For:

**Flowering → Pest Control**

Assume:

- Reward = 8
- Next state = Maturity
- Maximum future Q-value = 0

**Q-Table After Iteration 1**

| **State** | **Irrigate** | **Fertilize** | **Pest Control** | **No Action** |
|---|---|---|---|---|
| **Seedling** | **2.50** | **0** | **0** | **0** |

| | | | | |
|---|---|---|---|---|
| **Vegetative** | **0** | **3.00** | **0** | **0** |
| **Flowering** | **0** | **0** | **4.00** | **0** |
| **Maturity** | **0** | **0** | **0** | **0** |

**Iteration 2**
Now the agent uses the previously learned Q-values.
**State-Action Pair 1**
For:
**Seedling → Irrigate**
**State-Action Pair 2**
For:
**Vegetative → Fertilize**
**State-Action Pair 3**
For:
**Flowering → Pest Control**
**Q-Table After Iteration 2**

| State | Irrigate | Fertilize | Pest Control | No Action |
|---|---|---|---|---|
| **Seedling** | **5.10** | **0** | **0** | **0** |
| **Vegetative** | **0** | **6.30** | **0** | **0** |
| **Flowering** | **0** | **0** | **6.00** | **0** |
| **Maturity** | **0** | **0** | **0** | **0** |

**Training Episodes**
The agent is trained for **5 growing-season episodes**:
Episode 1 → Exploration
Episode 2 → Learning
Episode 3 → Improved decisions
Episode 4 → Policy stabilisation
Episode 5 → Final learned policy
**Convergence Criterion**
The Q-Learning process is considered converged when:
**The maximum change in Q-values between consecutive training episodes is less than 0.01 for two consecutive episodes.**
This indicates that the Q-table is becoming stable and additional training produces very little change.

**(c) Final Learned Policy**
The learned policy selects the action having the **highest Q-value for each state**.
Example final policy:
Therefore:

| Crop Growth Stage | Best action |
|---|---|
| Seedling | Irrigate |
| Vegetative | Fertilize |
| Flowering | Pest Control |
| Maturity | No Action |

Seedling
↓
Irrigate
↓
Vegetative
↓
Fertilize
↓
Flowering
↓
Pest Control
↓
Maturity
↓
No Action

**Ethical Considerations**

**1. Resource Overuse**

An RL agent may recommend excessive irrigation or fertilizer if the reward function is poorly designed. This can waste **water, fertilizer and energy**.

**Solution:** Include resource consumption and sustainability penalties in the reward function.

**2. Environmental Impact**

Excessive fertilizer or pesticide use can cause soil and water pollution.

**Solution:** Add environmental constraints and penalties for excessive interventions.

**3. Farmer Trust**

Farmers may not trust an AI system if it recommends an action without explaining why.

**Solution:** Provide understandable explanations such as:

"High soil moisture and sufficient rainfall → No irrigation recommended."

**4. Explainability**

RL decisions can be difficult to understand because the agent learns through repeated interactions.

**Solution:** Display the current state, recommended action, Q-value and reason for the recommendation.

**5. Human Oversight**

The system should act as a **decision-support tool**, not completely replace the farmer or agricultural expert. Important decisions should allow human review.

**Conclusion**

The farming problem can be modeled as an MDP with **crop-growth stages as states, four farming actions, yield improvement as the reward, and crop-stage progression as the transition dynamics**. Q-Learning is then used to learn the best action through at least five simulated growing-season episodes. The final policy selects the action with the highest Q-value. Ethical deployment requires controlling resource usage, reducing environmental impact, maintaining farmer trust, providing explanations, and keeping **human oversight** in the decision-making process.

## RUBRICS FOR CALCULATION:

| Criteria | Weigh tage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning | Marks |
|---|---|---|---|---|---|---|
| Understanding of Industry Problem | 20% | Comprehensive understanding of real-world problem and constraints | Good understanding with minor gaps in requirements | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context | |
| Application of Engineering Concepts | 25% | Applies advanced and relevant concepts effectively to solve the problem | Applies appropriate concepts with minor errors | Limited application of concepts | Incorrect or no application of concepts | |
| Solution Design | 25% | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps | Basic solution with limited feasibility | Unclear or impractical solution | |
| Use of Tools | 15% | Effective use of modern tools, technologies, and real data | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools | Minimal or incorrect use of tools | |
| Analysis | 10% | Thorough analysis with validated results and performance evaluation | Good analysis with reasonable validation | Basic analysis with limited validation | Poor or no analysis | |
| Professionalism | 5% | Highly professional, well-documented, and clearly presented work | Good documentation and presentation | Basic documentation with some gaps | Poor documentation and presentation | |